Serialization and Normalization - General & Project-Specific Guide

Table of Contents

---

1. Serialization (General)

What is Serialization?

Serialization is the process of converting an object/data structure into a format that can be:

- Stored (in files, databases)
- Transmitted (over network/API)
- Shared between systems

The reverse process is **deserialization** (converting back to usable objects).

Why Serialize?

```
Application Objects (in memory)
    ↓ SERIALIZE
Bytes/String/JSON (storable/transferable)
    ↓ SEND/STORE
Network/Disk
    ↓ RECEIVE/LOAD
Bytes/String/JSON
    ↓ DESERIALIZE
Application Objects (in memory)
```

Common Serialization Formats

Format	Use Case	Example
JSON	Web APIs, configs	`{"name": "John", "age": 30}`
XML	Legacy systems, SOAP	`<user><name>John</name></user>`
Protocol Buffers	High-performance systems	Binary format
MessagePack	Efficient binary	Compact JSON-like format
CSV	Tabular data	`name,age\nJohn,30`
Binary	Files, images	Raw bytes

Serialization Issues & Solutions

```
Problem: Circular References
Object A → Object B → Object A (infinite loop!)
Solution: Track visited objects, use references

Problem: Type Information Lost
Serialize: {value: 123} → Deserialize: is this int or string?
Solution: Include type hints in serialization

Problem: Date/Time Formatting
Date object → "2024-02-01" → Different formats in different systems
Solution: Use ISO 8601 standard (2024-02-01T12:30:45Z)

Problem: Large Objects
Serialize entire 1 million record dataset → Very large file
Solution: Stream serialization, pagination, compression
```

---

2. Normalization (General)

What is Normalization?

Normalization is organizing data to:

- Reduce redundancy
- Improve data integrity
- Enable efficient querying
- Avoid update anomalies

It's a database design technique to organize data into tables/relationships efficiently.

Levels of Normalization (Database)

1NF - First Normal Form

- No repeating groups
- Atomic values only

Bad (Unnormalized):

```
User: {
  id: 1,
  name: "John",
  hobbies: ["reading", "gaming"]  ← List in single field
}
```

Good (1NF):

```
Users:
  id | name
  1  | John

Hobbies:
  user_id | hobby
  1       | reading
  1       | gaming
```

2NF - Second Normal Form

- Satisfies 1NF
- Non-key attributes depend on entire primary key

Bad:

```
Orders:
  order_id | customer_id | customer_name | item_id | item_price
```

(customer_name depends only on customer_id, not on order_id)

Good:

```
Orders:
  order_id | customer_id | item_id

Customers:
  customer_id | customer_name

OrderItems:
  order_id | item_id | price
```

3NF - Third Normal Form

- Satisfies 2NF
- Non-key attributes don't depend on other non-key attributes

Bad:

```
Employees:
  emp_id | name | dept_id | dept_name | dept_location
```

(dept_name and dept_location depend on dept_id, not emp_id)

Good:

```
Employees:
  emp_id | name | dept_id

Departments:
  dept_id | dept_name | dept_location
```

Normalization vs Denormalization

Aspect	Normalized	Denormalized
Data Redundancy	Low	High
Query Speed	Slower (joins)	Faster (direct access)
Update Efficiency	Good	Poor (update many places)
Storage	Minimal	More
Use Case	OLTP (transactions)	OLAP (analytics)

---

3. Serialization in This Project

Overview

Your MetaDB project handles serialization at multiple levels:

```
 Application Layer
    ↓ to_dict() / to_json()
Python Objects (Models)
    ↓ JSON serialization
HTTP Response / Storage
```

How Serialization Works in MetaDB

3.1 Model Serialization Methods

Location: `flask_backend/app/models.py`

Each model has a `to_dict()` method for serialization:

```
# User Model
class User(db.Model):
    def to_dict(self):
        return {
            "id": self.id,
            "username": self.username,
            "email": self.email,
            "role": self.role
        }
```

Output: Python dict → Flask automatically converts to JSON

3.2 Schema Serialization

```
class SchemaModel(db.Model):
    def to_dict(self, include_fields=True):
        result = {
            "id": self.id,
            "name": self.name,
            "version": self.version,
            "asset_type_id": self.asset_type_id,
            "fields": []  # Optional nested serialization
        }
        if include_fields:
            # Serialize child objects
            result["fields"] = [field.to_dict() for field in self.fields]
        return result
```

Key Feature: Optional nested serialization (include_fields parameter)

3.3 Metadata Record Serialization

```
class MetadataRecord(db.Model):
    def to_dict(self, include_values=True):
        result = {
            "id": self.id,
            "name": self.name,
            "schema_id": self.schema_id,
            "created_at": self.created_at.isoformat(),  # ISO 8601 format
            "values": {}
        }

        if include_values:
            # Multiple storage types handled:
            # 1. EAV (Entity-Attribute-Value) - separate table
            if self.field_values:
                result["values"] = {
                    fv.schema_field.field_name: fv.get_value()
                    for fv in self.field_values
                }
                result["storage_type"] = "eav"

            # 2. JSON storage - legacy/bulk
            elif self.metadata_json:
                result["values"] = self.metadata_json
                result["storage_type"] = "json"

            # 3. Data rows table - new approach
            elif self.data_rows.count() > 0:
                result["row_count"] = self.data_rows.count()
                result["storage_type"] = "table"

        return result
```

Smart Serialization: Handles 3 different storage strategies transparently!

3.4 Marshmallow Schemas

Location: `flask_backend/app/schemas.py`

Marshmallow is used for validation + serialization:

```
from marshmallow import fields, validate

class MetadataRecordSchema(ma.Schema):
    id = fields.Int(dump_only=True)              # Only serialize, don't deserialize
    schema_id = fields.Int(required=True)        # Must have on input
    metadata_json = fields.Dict(required=True)   # Any dict is valid
    tag = fields.Str()                           # Optional string

    # Automatic serialization + validation
```

Usage:

```
schema = MetadataRecordSchema()
result = schema.dump(metadata_record_obj)  # Serialize + validate
```

3.5 Data Type Serialization in Routes

Location: `flask_backend/app/routes/metadata.py`

JSON parsing and type conversion:

```
@metadata_bp.route("/", methods=["POST"])
def create_metadata():
    # Deserialize JSON from request
    values = request.get_json()  # String → Python dict

    # Handle multiple input formats:
    if input_format == 'json':
        parsed = json.loads(raw_data)  # String → Object
    elif input_format == 'csv':
        # CSV → List of dicts
        reader = csv.reader(io.StringIO(raw_data))
    elif input_format == 'ndjson':
        # Newline-delimited JSON
        for line in raw_data.splitlines():
            parsed = json.loads(line)  # Line → Object

    # Serialize to storage
    record = MetadataRecord(metadata_json=values)  # Dict → JSON column
    db.session.add(record)
    db.session.commit()

    # Serialize response
    return jsonify(record.to_dict())  # Object → JSON response
```

3.6 DataRow Serialization

```
class DataRow(db.Model):
    data = db.Column(db.JSON, nullable=False)  # JSONB column

    def to_dict(self):
        return {
            "id": self.id,
            "record_id": self.record_id,
            "row_index": self.row_index,
            "data": self.data,  # Already dict, no parsing needed
            "created_at": self.created_at.isoformat()
        }
```

PostgreSQL JSONB: Stores as JSON, queryable like relational data

3.7 Frontend Serialization

Location: `Frontend/src/pages/Metadata.tsx`

TypeScript/React handling JSON:

```
// Deserialize API response
const response = await fetch('/api/metadata');
const data: MetadataRecord[] = await response.json();  // JSON → Objects

// Serialize for submission
const payload = {
  ...valuesToSend,
  // Type conversion for special fields:
  json_field: typeof raw === 'string' ? JSON.parse(raw) : raw,
  array_field: raw.split(',').map(v => v.trim())
};

// Send to API
await fetch('/api/metadata', {
  method: 'POST',
  body: JSON.stringify(payload)  // Objects → JSON
});
```

Serialization Flow in MetaDB

```
REQUEST (JSON from client)
    ↓
request.get_json() [Deserialize]
    ↓
Validate with Marshmallow schema
    ↓
Create model objects
    ↓
Save to database
    ↓
Query model objects
    ↓
model.to_dict() [Serialize]
    ↓
jsonify() [Convert dict to JSON]
    ↓
RESPONSE (JSON to client)
```

---

4. Normalization in This Project

Overview

Your MetaDB uses **multiple normalization strategies** to handle flexible dynamic schemas:

```
Traditional Normalization
    ↓
EAV (Entity-Attribute-Value) Model
    ↓
JSON Storage (JSONB in PostgreSQL)
    ↓
Hybrid Approach
```

4.1 Traditional Normalization

Schema Design

```
-- Normalized structure
users
  ├─ id (PK)
  ├─ username
  └─ email

schemas
  ├─ id (PK)
  ├─ name
  ├─ version
  ├─ asset_type_id (FK)
  └─ created_by (FK → users)

schema_fields
  ├─ id (PK)
  ├─ schema_id (FK)
  ├─ field_name
  ├─ field_type
  ├─ is_required
  └─ constraints (JSON)

metadata_records
  ├─ id (PK)
  ├─ schema_id (FK)
  ├─ asset_type_id (FK)
  ├─ created_by (FK)
  └─ metadata_json (JSON) or raw_data (JSON)
```

Benefits:

- ✅ Foreign keys enforce referential integrity
- ✅ Indexed lookups (schema_id, asset_type_id)
- ✅ Easy schema evolution (add/remove fields)
- ✅ Version tracking (version column)

Normalization in Models

```
# 1st Normal Form - Atomic values
class SchemaField(db.Model):
    field_name = db.String(128)      # Not a list, single value
    field_type = db.String(50)        # Single type per field
    is_required = db.Boolean()        # Atomic boolean
    constraints = db.JSON()           # Complex data in JSON

# 2nd Normal Form - Functional dependencies
class MetadataRecord(db.Model):
    schema_id = db.ForeignKey()       # Depends on entire PK
    asset_type_id = db.ForeignKey()   # Separate relationship
    created_by = db.ForeignKey()      # Who created it

# 3rd Normal Form - No transitive dependencies
# user_name doesn't depend on record_id, only on user_id
# Solution: Keep only user_id, join with users table for name
```

4.2 EAV (Entity-Attribute-Value) Model

Location: `FieldValue` model in models.py

```
class FieldValue(db.Model):
    """
    Entity-Attribute-Value pattern for handling dynamic schemas
    """
    record_id = db.Column(db.Integer, db.ForeignKey('metadata_records.id'))
    field_id = db.Column(db.Integer, db.ForeignKey('schema_fields.id'))

    # Store values in separate columns by type
    value_text = db.Column(db.String(1000))
    value_int = db.Column(db.Integer)
    value_float = db.Column(db.Float)
    value_bool = db.Column(db.Boolean)
    value_date = db.Column(db.Date)
    value_json = db.Column(db.JSON)
```

Why EAV?

Problem: Schema is dynamic - can't create column for every field

Traditional Approach (Fixed Schema):

```
metadata_records {
  id, name, email, age, phone
}
```

Problem: Must alter table to add new fields!

EAV Approach (Flexible):

```
metadata_records { id }
field_values {
  record_id, field_id, value_text (for "name")
  record_id, field_id, value_int (for "age")
  record_id, field_id, value_text (for "email")
}
```

Benefit: Add new fields without altering tables!

EAV Query Example

```
# Get all values for a record
def to_dict(self, include_values=True):
    if include_values:
        result["values"] = {
            fv.schema_field.field_name: fv.get_value()
            for fv in self.field_values
        }
```

Reconstructs:

```
{
  "name": "John",      # from value_text
  "age": 30,           # from value_int
  "active": True       # from value_bool
}
```

EAV Normalization Benefits

Aspect	Benefit
Flexibility	Add fields without schema migration
Storage	Sparse data (NULLs saved as missing rows)

Aspect	Benefit
Tracking	Each value can have timestamps
Validation	Enforce type per field
History	Easy to track field value changes

EAV Drawbacks

Issue	Solution in MetaDB
Slow queries (many joins)	Use JSONB for bulk data
Complex queries	Provide simplified API
Type checking	Separate columns per type
Aggregations	JSONB aggregation in PostgreSQL

4.3 JSONB Storage (Hybrid Normalization)

Location: `DataRow` model, bulk imports

```
class DataRow(db.Model):
    """
    Denormalized JSONB storage for better performance on bulk data
    PostgreSQL treats JSONB as semi-structured
    """
    record_id = db.Integer              # Normalized FK
    row_index = db.Integer              # Ordering (normalized)
    data = db.Column(db.JSON)           # Semi-denormalized JSON
```

Why JSONB?

Problem with Pure EAV:

- 10,000 records × 50 fields = 500,000 rows in FieldValue table
- Very slow to query all data for a record

JSONB Solution:

```
-- One row per record
data_rows {
  id: 1,
  record_id: 100,
  data: {
    "name": "John",
    "age": 30,
    "email": "john@example.com",
    "address": {
      "city": "NYC",
      "zip": "10001"
    }
  }
}
```

JSONB Benefits

```
 # PostgreSQL JSONB queries:
# Filter by nested field
SELECT * FROM data_rows WHERE data->>'city' = 'NYC'

# Index JSONB for fast lookups
CREATE INDEX idx_jsonb_city ON data_rows USING gin (data)
SELECT * FROM data_rows WHERE data @> '{"city": "NYC"}'

# Better than storing raw JSON string
```

4.4 Hybrid Normalization Strategy

MetaDB uses **three approaches** depending on use case:

```
 SCENARIO 1: Small single record
├─ Store in: metadata_json (simple)
└─ Query: Direct JSON access

SCENARIO 2: Medium record with tracking needs
├─ Store in: FieldValue table (EAV)
├─ Benefits: Individual field versioning, typed storage
└─ Query: Join on schema_field

SCENARIO 3: Large bulk data
├─ Store in: DataRow table (JSONB)
├─ Benefits: Fast access, natural grouping
└─ Query: PostgreSQL JSONB operators
```

Migration Between Storage Types

```
@metadata_bp.route("/<int:record_id>/migrate-now", methods=["POST"])
def manual_migrate_data(record_id):
    """
    Normalize data: JSON → EAV or JSON → DataRow
    """
    record = MetadataRecord.query.get(record_id)

    # Denormalize: Get data from JSON
    old_data = json.loads(record.metadata_json)

    # Normalize: Store in DataRow table
    for idx, row_data in enumerate(old_data):
        new_row = DataRow(
            record_id=record_id,
            row_index=idx,
            data=row_data
        )
        db.session.add(new_row)

    db.session.commit()
    # Result: Better indexed, faster queries
```

4.5 Referential Integrity (Normalization Constraint)

```
class MetadataRecord(db.Model):
    schema_id = db.ForeignKey(
        "schemas.id",
        ondelete="CASCADE",      # Delete all metadata if schema deleted
        onupdate="CASCADE"       # Update if schema ID changes
    )
    asset_type_id = db.ForeignKey(
        "asset_types.id",
        ondelete="SET NULL"      # Don't delete metadata, just clear ref
    )
```

Ensures:

- ✅ Can't create metadata for non-existent schema
- ✅ Can't have dangling references
- ✅ Cascading deletes work correctly

4.6 Data Consistency Patterns

Atomic Operations

```
 # Normalize: Store version when schema changes
@schemas_bp.route("/<int:schema_id>/fields", methods=["POST"])
def add_field(schema_id):
    schema = SchemaModel.query.get(schema_id)

    # Transaction: All or nothing
    db.session.begin_nested()
    try:
        # 1. Add field definition (normalized)
        field = SchemaField(schema_id=schema_id, ...)
        db.session.add(field)

        # 2. Increment version (normalized)
        schema.version += 1

        # 3. Record change (audit trail, normalized)
        changelog = ChangeLog(schema_id=schema_id, ...)
        db.session.add(changelog)

        db.session.commit()  # Atomic
    except:
        db.session.rollback()
```

Result: Normalized, consistent state

---

5. Real-World Examples

Example 1: Single Record (JSON Serialization)

```
 API Request:
POST /metadata
{
  "schema_id": 1,
  "values": {
    "name": "John Doe",
    "age": 30,
    "email": "john@example.com"
  }
}

Deserialization (JSON → Python):
request.get_json()
→ {"schema_id": 1, "values": {...}}

Store in Database:
MetadataRecord(
  schema_id=1,
  metadata_json={"name": "John Doe", "age": 30, ...}
)

Serialize Response:
record.to_dict()
→ {
  "id": 1,
  "schema_id": 1,
  "values": {"name": "John Doe", "age": 30, ...},
  "storage_type": "json"
}

API Response:
200 OK
{
  "id": 1,
  "schema_id": 1,
  "values": {...}
}
```

Example 2: Bulk Records (JSONB + Normalization)

```
 API Request (Bulk):
POST /metadata
{
  "schema_id": 1,
  "raw_data": "[
    {"name": "John", "age": 30},
    {"name": "Jane", "age": 28},
    {"name": "Bob", "age": 35}
  ]"
}

Deserialization:
json.loads(raw_data)
→ [{"name": "John", ...}, ...]

Normalize to DataRow table:
for idx, row in enumerate(data):
  DataRow(record_id=1, row_index=idx, data=row)

Database:
metadata_records
  id: 1, name: "Bulk Import", schema_id: 1

data_rows
  id: 1, record_id: 1, row_index: 1, data: {"name": "John", "age": 30}
  id: 2, record_id: 1, row_index: 2, data: {"name": "Jane", "age": 28}
  id: 3, record_id: 1, row_index: 3, data: {"name": "Bob", "age": 35}

Serialize Response:
record.to_dict(include_values=True)
→ {
  "id": 1,
  "row_count": 3,
  "is_bulk_import": true,
  "storage_type": "table"
}
```

Example 3: Type Conversion (Serialization)

```
Frontend Input:
{
  "price": "99.99",      # String from form
  "tags": "python,flask", # String from form
  "active": "true"        # String from checkbox
}

Serialization/Type Conversion:
const transformed = {
  price: parseFloat("99.99"),           // String → float
  tags: "python,flask".split(','),      // String → array
  active: "true" === "true"             // String → boolean
}

Send to Backend:
JSON.stringify(transformed)
→ {
  "price": 99.99,
  "tags": ["python", "flask"],
  "active": true
}

Backend Validation:
Marshmallow schema checks types
✓ price is number
✓ tags is array
✓ active is boolean

Store:
metadata_json={price: 99.99, tags: [...], active: true}

Retrieve & Serialize:
to_dict() returns properly typed values
→ Client receives typed data
```

Example 4: Schema Evolution (Normalization)

```
Original Schema (v1):
field_definitions:
  - name: String
  - age: Integer

Add new field (v2):
POST /schemas/1/fields
{
  "name": "email",
  "type": "string",
  "required": false
}

Normalized Storage:
schema_model: version = 2
schema_fields: Add new row (email field)
schema_version: Create snapshot

No data migration needed!
✓ Existing records have email=NULL (sparse)
✓ New records can have email value
✓ Full backward compatibility

Query impact:
- EAV: Add row in FieldValue table when email provided
- JSONB: Add property when provided
- Both: Graceful handling of missing values
```

Summary

Serialization in MetaDB

- **Purpose**: Convert Python objects ↔ JSON for storage/transmission
- **Methods**: `to_dict()`, Marshmallow schemas
- **Formats**: JSON, CSV, NDJSON, JSONB
- **Challenge**: Handle multiple storage types transparently

Normalization in MetaDB

- **Purpose**: Organize data to reduce redundancy, improve integrity
- **Approach**: Hybrid (EAV + JSONB + Traditional)
- **Flexibility**: Support dynamic schemas without migration
- **Trade-off**: Performance vs flexibility

Key Takeaway

MetaDB normalizes **structure** (schemas, relationships) while keeping **data flexible** (JSON/JSONB storage), achieving both data integrity and schema flexibility.